Recipe 13: Creating and Deploying a Simple Infrastructure using Terraform Objective

Create main.tf

```
provider "aws"{
        region="ap-south-1"
}


resource "aws_s3_bucket" "my_bucket"{
        bucket="myuniquet2341"
}
```

Recipe 14: Managing Terraform State in a Terraform Objective

Step 1: create s3 bucket

Step 2: write this script in main.tf

```
terraform {
  backend "s3" {
    bucket = "my-uniquebuckett-312"
    key    = "terraform-s3/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

Recipe 15: Provisioning a Virtual Machine using Terraform Objective

```
provider "aws"{
     region="ap-south-1"
}


resource "aws_instance" "my_ec2"{
    ami="ami-0e12ffc2dd465f6e4"
    instance_type="t3.micro"  #take ami id from ec2->create instance->select amazon/linux
    key_name="my-instance"   #Open EC2 Dashboard   In left menu → click Key Pairs
```

```
}
```

Recipe 16: Provisioning a Web Server using Terraform Objective

```
provider "aws"{
    region="ap-south-1"
}

resource "aws_security_group" "web_sg"{
  ingress{
            from_port=22
            to_port=22
            protocol="tcp"
            cidr_blocks=["0.0.0.0/0"]
  }

  ingress{
            from_port=80
            to_port=80
            protocol="tcp"
            cidr_blocks=["0.0.0.0/0"]
  }

}
resource "aws_instance" "my_ec2"{
   ami="ami-0e12ffc2dd465f6e4"
   instance_type="t3.micro"  #take ami id from ec2->create instance->select amazon/linux
```

```
  key_name="my-instance"   #Open EC2 Dashboard   In left menu → click Key Pairs


  security_groups = [aws_security_group.web_sg.name]


user_data=<<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
echo "hello world"> /var/www/html/index.html
EOF
}
```

For run:

Go to ec2 -> instance ->copy public Ipv4 address

Recipe 17: Provisioning a Database using Terraform Objective

```
provider "aws" {
  region = "ap-south-1"
}


resource "aws_db_instance" "mydb" {
  allocated_storage    = 20
  engine               = "mysql"
  instance_class       = "db.t3.micro"
  db_name              = "mydb"
```

```
  username            = "admin"
  password            = "Admin1234"
  publicly_accessible  = true
  skip_final_snapshot  = true
}
```

Run:

Amazon rds->databases

Recipe 18: Deploying a Web Application using Terraform Objective

```
provider "aws" {
  region = "ap-south-1"
}

# ------------------ Security Group for EC2 ------------------
resource "aws_security_group" "web_sg" {
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------ Security Group for RDS ------------------
resource "aws_security_group" "db_sg" {
  ingress {
```

```
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.web_sg.id]
  }
}

# ------------------ EC2 Instance ------------------
resource "aws_instance" "web" {
  ami           = "ami-0e12ffc2dd465f6e4"
  instance_type = "t2.micro"
  key_name      = "my-instance"

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
echo "Web Server Connected to DB" > /var/www/html/index.html
EOF
}

# ------------------ RDS Database ------------------
resource "aws_db_instance" "db" {
  allocated_storage    = 20
  engine               = "mysql"
```

```
  instance_class       = "db.t3.micro"

  db_name              = "mydb"

  username             = "admin"

  password             = "Admin1234"


  vpc_security_group_ids = [aws_security_group.db_sg.id]


  publicly_accessible  = true

  skip_final_snapshot  = true

}
```

How to run:

Ec2->copy public ip ->http:// ip

Recipe 19: Managing Infrastructure Changes using Terraform Objective

Old code:

```
provider "aws"{

    region="ap-south-1"

}


resource "aws_instance" "my_ec2"{

   ami="ami-0e12ffc2dd465f6e4"

   instance_type="t3.micro"  #take ami id from ec2->create instance->select amazon/linux

   key_name="my-instance"   #Open EC2 Dashboard   In left menu → click Key Pairs

}
```

Plan

Apply

New code:

```
provider "aws"{

    region="ap-south-1"

}



resource "aws_instance" "my_ec2"{

   ami="ami-0e12ffc2dd465f6e4"

   instance_type="t3.micro"  #take ami id from ec2->create instance->select amazon/linux

   key_name="my-instance"   #Open EC2 Dashboard   In left menu → click Key Pairs



   tags = {

   Name = "Updated-Instance"

  }

}
```

```
terraform plan
```

You should see:

```
~ tags: Name = "old" -> "Updated-Instance"
```

```
Recipe 20: Using Terraform to Configure Load Balancers 0Objective
provider "aws" {
  region = "ap-south-1"
}

# ------------------ Default VPC ------------------
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# ------------------ Security Group ------------------
resource "aws_security_group" "sg" {
```

```
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------ EC2 Instance 1 ------------------
resource "aws_instance" "web1" {
  ami           = "ami-0e12ffc2dd465f6e4"
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default.ids[0]

  vpc_security_group_ids = [aws_security_group.sg.id]

  user_data = <<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
echo "Server 1" > /var/www/html/index.html
EOF
}

# ------------------ EC2 Instance 2 ------------------
resource "aws_instance" "web2" {
  ami           = "ami-0e12ffc2dd465f6e4"
  instance_type = "t3.micro"
  subnet_id     = data.aws_subnets.default.ids[1]

  vpc_security_group_ids = [aws_security_group.sg.id]

  user_data = <<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
echo "Server 2" > /var/www/html/index.html
EOF
}

# ------------------ Target Group ------------------
resource "aws_lb_target_group" "tg" {
  port     = 80
  protocol = "HTTP"
  vpc_id   = data.aws_vpc.default.id
```

```
}

# ------------------ Load Balancer ------------------
resource "aws_lb" "alb" {
  load_balancer_type = "application"
  subnets            = data.aws_subnets.default.ids
  security_groups    = [aws_security_group.sg.id]
}

# ------------------ Listener ------------------
resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}

# ------------------ Attach EC2 to Target Group ------------------
resource "aws_lb_target_group_attachment" "t1" {
  target_group_arn = aws_lb_target_group.tg.arn
  target_id        = aws_instance.web1.id
  port             = 80
}

resource "aws_lb_target_group_attachment" "t2" {
  target_group_arn = aws_lb_target_group.tg.arn
  target_id        = aws_instance.web2.id
  port             = 80
}

# ------------------ Output ------------------
output "alb_dns" {
  value = aws_lb.alb.dns_name
}
```

How to run:
After apply cmd generate link copy and paste output is server 1 serevr2

Recipe 21: Managing Infrastructure Dependencies with Terraform

```
provider "aws" {
  region = "ap-south-1"
}

# ------------------ VPC ------------------
resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"
}

# ------------------ Internet Gateway ------------------
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.my_vpc.id
}
```

```
# ------------------ Subnet ------------------
resource "aws_subnet" "my_subnet" {
  vpc_id                  = aws_vpc.my_vpc.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

# ------------------ Route Table ------------------
resource "aws_route_table" "rt" {
  vpc_id = aws_vpc.my_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

# ------------------ Route Table Association ------------------
resource "aws_route_table_association" "rta" {
  subnet_id      = aws_subnet.my_subnet.id
  route_table_id = aws_route_table.rt.id
}

# ------------------ Security Group ------------------
resource "aws_security_group" "sg" {
  name   = "my-sg"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }


  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------ EC2 Instance ------------------
resource "aws_instance" "web" {
  ami           = "ami-0e12ffc2dd465f6e4"
  instance_type = "t3.micro"

  subnet_id              = aws_subnet.my_subnet.id
  vpc_security_group_ids = [aws_security_group.sg.id]

  depends_on = [aws_internet_gateway.igw]
  tags = {
```

```
    Name = "Dependency-Example"
  }
}
```

Recipe 22: Using Terraform for Zero-Downtime Deployment

```
provider "aws" {
  region = "ap-south-1"
}

# ------------------ Default VPC ------------------
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# ------------------ Security Group ------------------
resource "aws_security_group" "sg" {
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------ Launch Template ------------------
resource "aws_launch_template" "lt" {
  name_prefix   = "web-"
  image_id      = "ami-0e12ffc2dd465f6e4"
  instance_type = "t3.micro"

  user_data = base64encode(<<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
echo "Version 1" > /var/www/html/index.html
EOF
  )

  vpc_security_group_ids = [aws_security_group.sg.id]
}

# ------------------ Target Group ------------------
```

```
resource "aws_lb_target_group" "tg" {
  port     = 80
  protocol = "HTTP"
  vpc_id   = data.aws_vpc.default.id
}

# ------------------ Load Balancer ------------------
resource "aws_lb" "alb" {
  load_balancer_type = "application"
  subnets            = data.aws_subnets.default.ids
}

# ------------------ Listener ------------------
resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}

# ------------------ Auto Scaling Group ------------------
resource "aws_autoscaling_group" "asg" {
  desired_capacity    = 2
  max_size            = 3
  min_size            = 2
  vpc_zone_identifier = data.aws_subnets.default.ids

  launch_template {
    id      = aws_launch_template.lt.id
    version = "$Latest"
  }

  target_group_arns = [aws_lb_target_group.tg.arn]


  lifecycle {
    create_before_destroy = true
  }
}
```

Run:
Load balancer->scroll down->dns name copy and paste url to browser

Recipe 23: Scaling Infrastructure using Terraform

```
provider "aws" {
  region = "ap-south-1"
}

# ------------------ Default VPC ------------------
data "aws_vpc" "default" {
  default = true
}


data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
```

```
  }
}

# ------------------ Security Group ------------------
resource "aws_security_group" "sg" {
  vpc_id = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ------------------ Launch Template ------------------
resource "aws_launch_template" "lt" {
  image_id      = "ami-0e12ffc2dd465f6e4"
  instance_type = "t2.micro"

  user_data = base64encode(<<EOF
#!/bin/bash
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo "Auto Scaling" > /var/www/html/index.html
EOF
  )

  vpc_security_group_ids = [aws_security_group.sg.id]
}

# ------------------ Target Group ------------------
resource "aws_lb_target_group" "tg" {
  port     = 80
  protocol = "HTTP"
  vpc_id   = data.aws_vpc.default.id
}

# ------------------ Load Balancer ------------------
resource "aws_lb" "alb" {
  load_balancer_type = "application"
  subnets            = data.aws_subnets.default.ids
  security_groups    = [aws_security_group.sg.id]
}

# ------------------ Listener ------------------
resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
```

```
  }
}

# ------------------ Auto Scaling Group ------------------
resource "aws_autoscaling_group" "asg" {
  desired_capacity    = 2
  max_size            = 3
  min_size            = 1
  vpc_zone_identifier = data.aws_subnets.default.ids

  launch_template {
    id      = aws_launch_template.lt.id
    version = "$Latest"
  }

  target_group_arns = [aws_lb_target_group.tg.arn]
}

# ------------------ Output ------------------
output "alb_dns" {
  value = aws_lb.alb.dns_name
}
```

How to run:
After apply cmd print one link open it and run